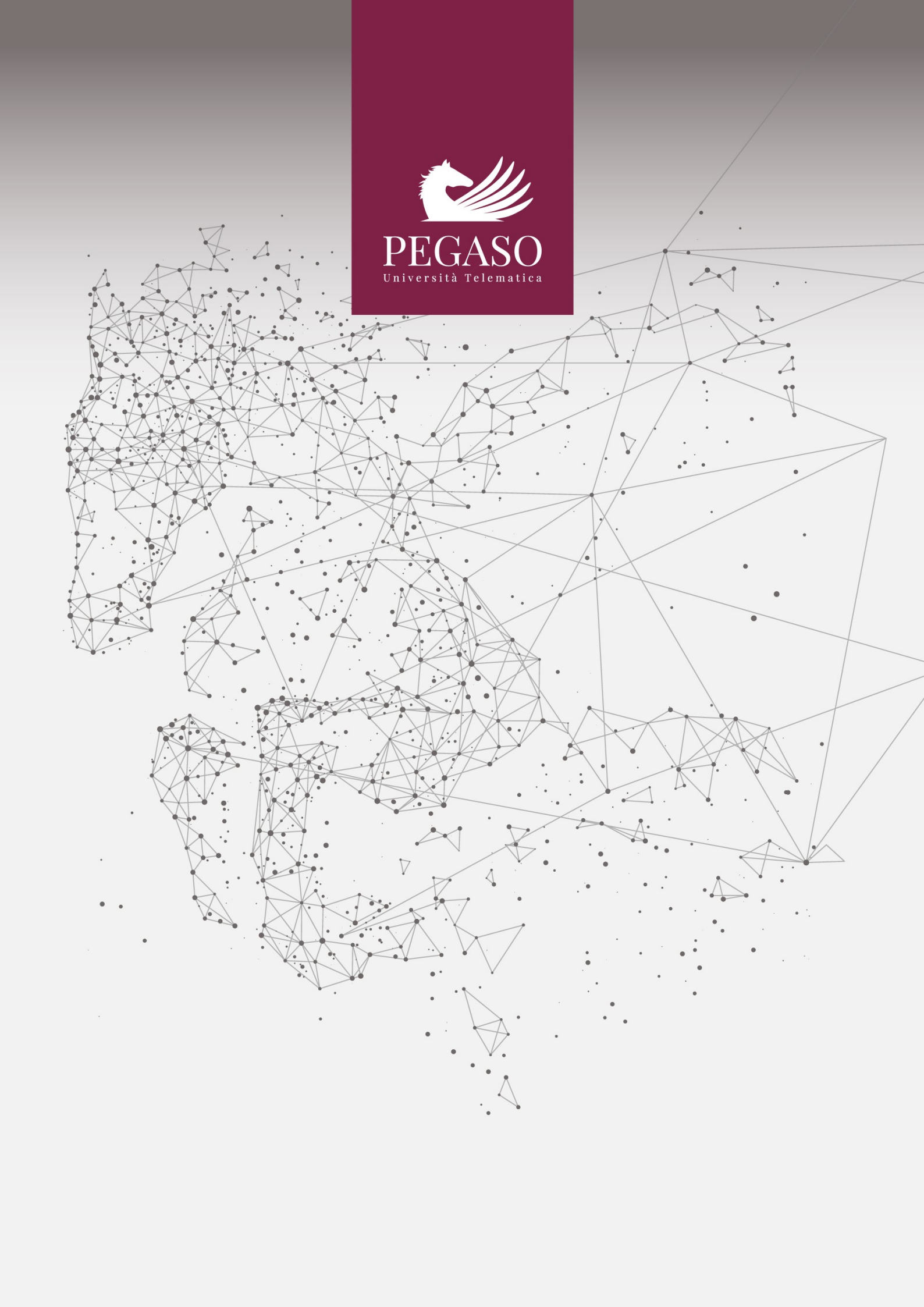

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. NUMERO PRIMO....4
3. SEQUENZA DI NUMERI PRIMI....7
4. OVERLOADING DELLA FUNZIONE SOMMA....9
5. DISTANZA SUL PIANO CARTESIANO....12
6. CALCOLO DEL PERIMETRO DI UN POLIGONO....15
BIBLIOGRAFIA....18

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si illustrano ed analizzano alcune esercitazioni molto importanti che approfondiscono il concetto di funzione ed i vantaggi nell'uso delle funzioni. In particolare, si riportano cinque esercizi in cui sono presenti delle funzioni definite dal programmatore. I primi due esercizi mostrano una funzione con parametro numerico e risultato binario. Il terzo esercizio mostra come Python implementa l'overloading delle funzioni. Il quarto e il quinto esercizio illustrano una funzione con più parametri numerici per la risoluzione di problemi geometrici.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Numero primo

In matematica, un numero primo è un numero intero positivo che abbia esattamente due divisori distinti. In modo equivalente si può definire come un numero naturale maggiore di 1 che sia divisibile solamente per 1 e per sé stesso.

Sulla base della definizione, è possibile definire una funzione che verifica se un numero è primo o no. In particolare:

- dato il numero n da esaminare passato come parametro alla funzione;
- si procede all'analisi del resto della divisione intera tra n e tutti i numeri compresi tra 2 e n-1 (quindi escludendo 1 e n che sono sicuramente divisori di n);
- nel caso in cui nessuno dei numeri compresi tra 2 e n-1 fornisse un resto pari a zero nella divisione intera con q, il numero si può considerare primo.

Per velocizzare l'elaborazione della funzione non si verifica che tutti i numeri compresi tra 2 e n-1 siano divisori di n. La verifica del numero n può essere interrotta non appena viene trovato il primo divisore compreso tra 2 e n-1.

Il codice Python che segue implementa l'algoritmo appena descritto. La struttura prevede prima la definizione della funzione primo che accetta un parametro n e, di seguito, il codice della funzione main che richiama la funzione primo(). Si noti che la funzione main deve essere richiamata al termine del codice per poter iniziare l'elaborazione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def primo(n):
    div=2
    while div<n:
        if n%div==0:
            return False
        div+=1
    return True


def main():
    print("NUMERO PRIMO")

    num=0
    while num<=1:
        num=int(input("Inserire un numero: "))

    if primo(num):
        print(num,"è un numero primo")
    else:
        print(num,"non è un numero primo")


main()
```

Di seguito si riporta il risultato dell'elaborazione del codice precedente per due casi: 67 (primo) e 18 (non primo).

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
NUMERO PRIMO
Inserire un numero: 67
67 è un numero primo
>>>
NUMERO PRIMO
Inserire un numero: 18
18 non è un numero primo
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Sequenza di numeri primi

Un esercizio interessante che sfrutta la funzione primo() per semplificare il processo di verifica è quello della sequenza dei numeri primi tra 1 e n. In particolare, dato un numero n, si vuole ottenere l'elenco di tutti i numeri primi tra 1 e n.

Sarà quindi necessario «produrre» tutti i numeri da 1 a n e verificare per ciascuno di essi il risultato restituito dalla funzione primo(). Per velocizzare l'elaborazione, si stampano nell'elenco dei numeri prima direttamente i numeri 1 e 2 e, successivamente, si procede ad esaminare i numeri a partire da 3 e incrementando ciascuna volta per 2 proprio per evitare di verificare i numeri pari che sono sicuramente divisibili per 2.

Il codice Python che segue implementa l'algoritmo appena descritto. La struttura prevede prima la definizione della funzione primo che accetta un parametro n e, di seguito, il codice della funzione main che richiama diverse volte la funzione primo() evidenziando la minore complessità del codice totale per verificare se un numero sia primo o meno. Si noti che la funzione main deve essere richiamata al termine del codice per poter iniziare l'elaborazione.

```
def primo(n):
    div=2
    while div<n:
        if n%div==0:
            return False
        div+=1
    return True


def main():
    print("SEQUENZA NUMERI PRIMI")

    num=0
    while num<=3:
        num=int(input("Fino a che numero vuoi elencare i numeri primi? "))

    print("Elenco numeri primi da 1 a",num)
    print("1, 2",end=", ")
    for i in range (3,num+1,2):
        if primo(i):
            print(i,end=", ")


main()
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Di seguito si riporta il risultato dell'elaborazione del codice precedente per n = 100.

```
SEQUENZA NUMERI PRIMI

Fino a che numero vuoi elencare i numeri primi?
100

Elenco numeri primi da 1 a 100

1, 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37,
41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89,
97
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Overloading della funzione somma

In programmazione l'overloading (o sovraccarico) è la creazione di una famiglia di funzioni/subroutine aventi tutte lo stesso nome, ma che accettano un diverso insieme di argomenti.

Purtroppo, non tutti i linguaggi di programmazione consentono l'overloading.

Python si comporta in modo molto particolare in riferimento all'overloading di funzioni. L'overloading è permesso solo in fase di definizione:

- la funzione viene definita secondo l'ultima definizione che si riporta in ordine sequenziale nel codice, di fatto sovrascrivendo le definizioni precedenti della stessa funzione;
- in fase di chiamata, quindi, un numero di parametri diverso da quello riportato nell'ultima definizione della funzione genera un'eccezione (errore a run-time).

Il codice Python che segue definisce prima la funzione somma() con tre parametri e poi ne effettua l'overloading definendo la funzione somma() con due parametri. Successivamente, richiama prima la funzione somma() con tre parametri e poi la funzione somma() con due parametri.

```
#eccezione nell'overloading - sovrascrittura di funzione

def somma(n1,n2,n3):
    return n1+n2+n3

def somma(n1,n2):
    return n1+n2


def main():
    print("OVERLOADING FUNZIONE")

    print("La somma tra 2,4 e 6 è:",somma(2,4,6))
    print("La somma tra 2 e 6 è:",somma(2,4))

main()
```

Di seguito si riporta il risultato dell'elaborazione del codice precedente che definisce come ultima la funzione somma con due parametri e richiama come prima la funzione somma a tre parametri generando un'eccezione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
OVERLOADING FUNZIONE
Traceback (most recent call last):
  File
        overloading_somma.py", line 17, in <module>
    main()
  File
        overloading_somma.py", line 14, in main
    print("La somma tra 2,4 e 6 è:",somma(2,4,6))
TypeError: somma() takes 2 positional arguments but 3 were given
```

Il codice Python che segue definisce prima la funzione somma() con tre parametri e poi ne effettua l'overloading definendo la funzione somma() con due parametri. Successivamente, richiama prima la funzione somma() con due parametri e poi la funzione somma() con tre parametri.

```
#eccezione nell'overloading - sovrascrittura di funzione

def somma(n1,n2,n3):
    return n1+n2+n3

def somma(n1,n2):
    return n1+n2


def main():
    print("OVERLOADING FUNZIONE")

    print("La somma tra 2 e 6 è:",somma(2,4))
    print("La somma tra 2,4 e 6 è:",somma(2,4,6))

main()
```

Di seguito, invece, si riporta il risultato dell'elaborazione del codice precedente che definisce come ultima la funzione somma con due parametri e richiama come prima la funzione somma a due parametri che viene richiamata in modo corretto. Successivamente viene chiamata la funzione somma con tre parametri che solleva un'eccezione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
OVERLOADING FUNZIONE
La somma tra 2 e 6 è: 6
Traceback (most recent call last):
  File "
        overloading_somma_invertito.py", line 17, in <module>
    main()
  File
        overloading_somma_invertito.py", line 15, in main
    print("La somma tra 2,4 e 6 è:",somma(2,4,6))
TypeError: somma() takes 2 positional arguments but 3 were given
```

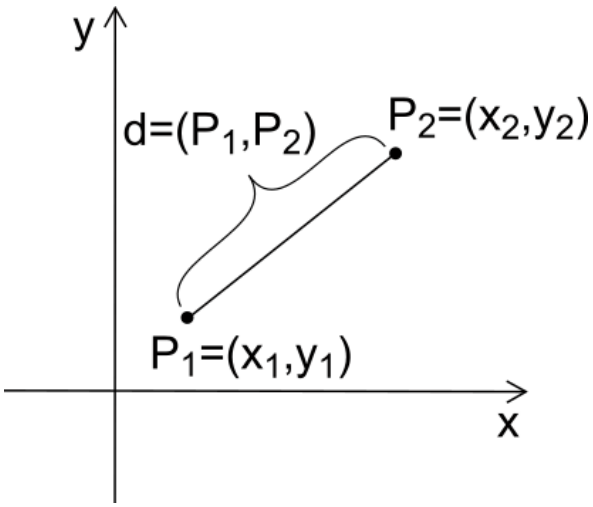
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Distanza sul piano cartesiano

La distanza tra due punti nel piano cartesiano (o distanza euclidea) si calcola come radice quadrata della somma tra il quadrato delle differenze delle ascisse e il quadrato della differenza delle ordinate dei due punti. La distanza tra due punti ciò è per definizione non negativa, quindi, è positiva oppure nulla se i due punti coincidono.

Con riferimento alla figura successiva, la formula per il calcolo della distanza tra due punti è la seguente:

$$d(P_1, P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



Il codice Python che segue importa la libreria math per poter utilizzare la funzione math.sqrt() per il calcolo della radice quadrata. Successivamente, definisce prima la funzione distanza() che accetta quattro parametri corrispondenti all'ascissa e all'ordinata dei due punti tra cui calcolare la somma e restituisce la misura della distanza tra i due parametri.

Infine, si definisce la funzione main che permette l'inserimento in input delle coordinate dei due punti e il calcolo della distanza tra essi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import math

def distanza(x1,y1,x2,y2):
    return math.sqrt(pow(x1-x2,2)+pow(y1-y2,2))


def main():
    print("DISTANZA TRA PUNTI")

    x1 = float(input("Inserire x1: "))
    y1 = float(input("Inserire y1: "))
    x2 = float(input("Inserire x2: "))
    y2 = float(input("Inserire y2: "))
    print("La distanza tra i due punti è:",distanza(x1,y1,x2,y2))


main()
```

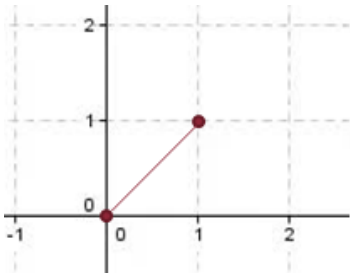
Di seguito si riporta il risultato dell'elaborazione del codice precedente per due casi: distanza tra (0,0) e (1,1) e distanza tra (1,1) e (2,2).

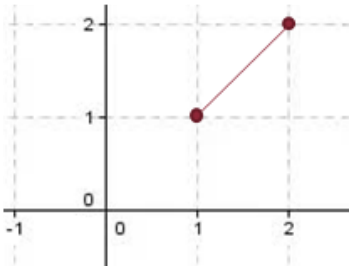
```
DISTANZA TRA PUNTI
Inserire x1: 0
Inserire y1: 0
Inserire x2: 1
Inserire y2: 1
La distanza tra i due punti è: 1.4142135623730951
>>>
DISTANZA TRA PUNTI
Inserire x1: 1
Inserire y1: 1
Inserire x2: 2
Inserire y2: 2
La distanza tra i due punti è: 1.4142135623730951
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'immagine successiva riporta la rappresentazione grafica sul piano cartesiano degli esempi di calcolo delle distanze tra due punti.





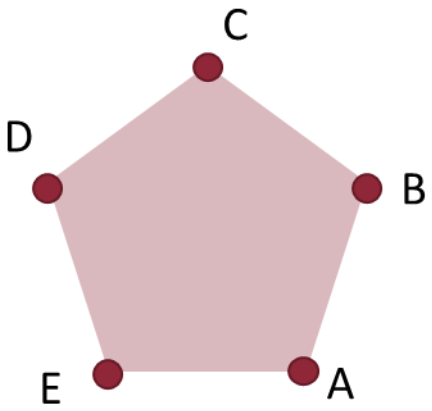
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

6. Calcolo del perimetro di un poligono

Un esercizio interessante che sfrutta la funzione distanza() è quello del calcolo del perimetro di un poligono.

Con riferimento alla figura successiva, il perimetro può essere, infatti, calcolato attraverso la distanza di due vertici contigui:

$$\overline{AB}, \overline{BC}, \overline{CD}, \overline{DE}, \overline{EA}$$



Il codice Python che segue importa la libreria math per poter utilizzare la funzione math.sqrt() per il calcolo della radice quadrata. Successivamente, definisce prima la funzione distanza() che accetta quattro parametri corrispondenti all'ascissa e all'ordinata dei due punti tra cui calcolare la somma e restituisce la misura della distanza tra i due parametri.

Si definisce, quindi, la funzione main che richiede inizializza il perimetro (perim) a zero e, attraverso un ciclo di validazione dell'input, richiede il numero dei vertici del poligono. Si provvede all'inserimento delle coordinate del primo punto e, poi, con un ciclo for si continua con l'inserimento dei vertici del poligono e il calcolo delle distanze tra i vertici a due a due, distanze che vengono sommate nella variabile associata al perimetro.

È fondamentale sottolineare che sono necessarie sia due coppie di coordinate per memorizzare i dati del vertice precedente e di quello successivo, sia le coordinate del primo punto inserito (nella figura precedente ad es. vertice A) per permettere il calcolo dell'ultima distanza (lato) tra l'ultimo punto inserito (nella figura precedente vertice E) e A e completare il calcolo del perimetro del poligono.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import math

def distanza(x1,y1,x2,y2):
    return math.sqrt(pow(x1-x2,2)+pow(y1-y2,2))


def main():
    print("PERIMETRO POLIGONO")

    perim = 0
    n=0

    while n<=2:
        n = int(input("Da quanti punti è composto il poligono? "))


    print("Inserire le coordinate dei punti")
```

```
    xp = float(input("Inserire coordinata x del 1o punto: "))
    yp = float(input("Inserire coordinata y del 1o punto: "))

    x0 = xp
    y0 = yp

    for i in range(2,n+1):
        xs = float(input("Inserire coordinata x del "+str(i)+"o punto: "))
        ys = float(input("Inserire coordinata y del "+str(i)+"o punto: "))
        perim += distanza(xp,yp,xs,ys)
        xp = xs
        yp = ys

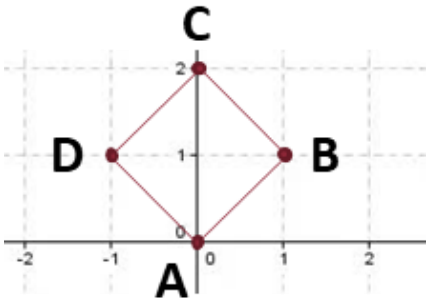
    perim+=distanza(xp,yp,x0,y0)

    print("Il perimetro del poligono di "+str(n)+" lati è pari a",perim)


main()
```

A titolo di esempio si consideri il quadrato ruotato e riportato nella figura successiva avente vertici nei punti A(0,0), B(1,1), C(0,2), D(-1,1).

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



Di seguito si riporta il risultato dell'elaborazione del codice precedente per il quadrato riportato nella figura precedente.

```
PERIMETRO POLIGONO
Da quanti punti è composto il poligono? 4
Inserire le coordinate dei punti
Inserire coordinata x del 1o punto: 0
Inserire coordinata y del 1o punto: 0
Inserire coordinata x del 2o punto: 1
Inserire coordinata y del 2o punto: 1
1.4142135623730951
Inserire coordinata x del 3o punto: 0
Inserire coordinata y del 3o punto: 2
1.4142135623730951
Inserire coordinata x del 4o punto: -1
Inserire coordinata y del 4o punto: 1
1.4142135623730951
Il perimetro del poligono di 4 lati è pari a
5.656854249492381
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).